DuocUC

El Almendro | ECO-HOTEL

EVALUACIÓN PARCIAL N°2

Desarrollo de un Agente Funcional

Proyecto: Hotel El Almendro

Carrera: Ingeniería en Informática
Asignatura: Ingeniería de Soluciones con Inteligencia Artificial
Equipo de Proyecto:
Matias Ignacio Loyola Cornejo - Matias Manuel Cisternas Stuardo
Docente: Michael Palomino

Fecha: 22 de Octubre de 2026

Resumen

El presente informe documenta la segunda etapa del proyecto de Ingeniería de Soluciones con IA para Hotel El Almendro. Se implementó un agente funcional orientado al área de Obras, Mantención y Mejoras (OMM), capaz de consultar archivos PDF y Excel, recuperar contexto mediante embeddings, seleccionar herramientas según la consulta, mantener memoria y generar salidas escritas. El prototipo fue desarrollado en Python y Google Colab, utilizando Gemini para generación de respuestas, gemini-embedding-2 para recuperación semántica y LangGraph para la orquestación. En la validación final se procesaron dos planillas Excel, generando 16 fragmentos indexados, y se ejecutaron siete casos de prueba con la herramienta esperada en todos los escenarios.

1. Diseño e implementación del agente

La solución organiza el procesamiento documental y la ejecución del agente en componentes independientes. Los archivos se cargan desde Colab, se normalizan y se dividen en fragmentos. Cada fragmento conserva el nombre del archivo y la hoja o página de origen. La consulta del usuario es analizada por un planificador que selecciona una ruta del grafo y ejecuta la herramienta correspondiente.

Componente	Implementación	Función
Entorno	Python + Google Colab	Ejecución del notebook y manejo seguro de credenciales con Secrets.
Modelo	gemini-3.5-flash-lite	Generación de respuestas y decisiones estructuradas en JSON.
Embeddings	gemini-embedding-2, 768 dimensiones	Representación semántica de documentos y consultas.
Fuentes	PDF, XLSX y XLSM	Lectura mediante PyPDF, Pandas y OpenPyXL.
Recuperación	Similitud coseno, TOP_K=5	Selección de fragmentos relevantes conservando la fuente.
Orquestación	LangGraph	Secuencia de nodos, rutas condicionales y estado del agente.

1.1 Herramientas

El agente integra seis acciones: buscar documentos, listar fuentes, analizar documentos, guardar memoria, generar reportes y responder desde memoria o contexto general. La herramienta de escritura crea archivos de reporte en formato TXT y la herramienta de análisis compara únicamente la información recuperada desde las fuentes cargadas.

2. Memoria y recuperación de contexto

La memoria de corto plazo conserva los últimos seis turnos mediante una cola de tamaño limitado. La memoria persistente almacena datos explícitamente solicitados por el usuario en un archivo JSON durante la sesión. Para la recuperación semántica, los fragmentos documentales y la consulta se transforman en embeddings de 768 dimensiones; posteriormente se aplica similitud coseno para seleccionar hasta cinco fragmentos relevantes.

Proceso	Implementación	Resultado
Corto plazo	deque(maxlen=6)	Mantiene continuidad en preguntas consecutivas.
Largo plazo	memoria_omm.json	Conserva datos marcados explícitamente para recordar.
Contexto semántico	Índice de embeddings	Recupera información relacionada aunque no coincidan exactamente las palabras.
Trazabilidad	Archivo + hoja/página	Permite indicar la fuente utilizada en la respuesta.

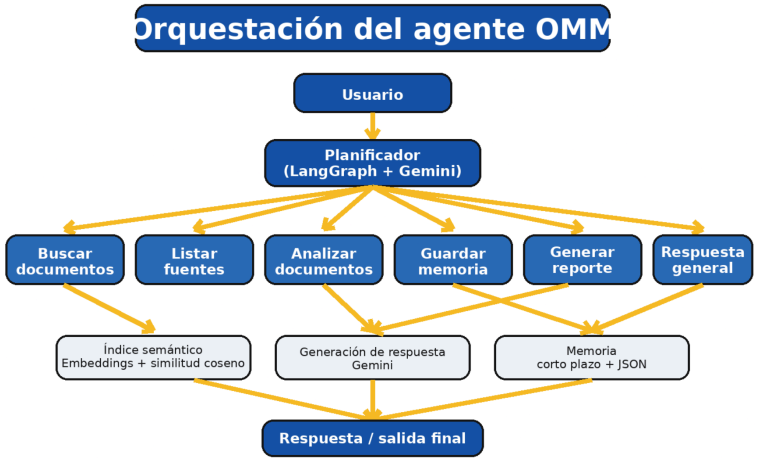


Figura 1. Flujo de orquestación implementado con LangGraph.

3. Diseño e implementación del pipeline RAG

El planificador recibe la consulta y el contexto de memoria y devuelve una decisión estructurada. La herramienta seleccionada depende del objetivo de la pregunta: recuperación factual, listado de fuentes, análisis comparativo, almacenamiento de memoria, generación de un reporte o respuesta general. Esta decisión determina la ruta que ejecuta el grafo.

3.1 Casos de prueba

Caso	Consulta / objetivo	Ruta esperada	Resultado
1	Proveedores que venden porcelanatos	buscar_documentos	CORRECTO
2	Productos de Cabaña Sol comprados en Easy	buscar_documentos	CORRECTO
3	Documentos y hojas disponibles	listar_documentos	CORRECTO
4	Proveedor con comentario más negativo	analizar_documentos	CORRECTO
5	Recordar proveedor prioritario	guardar_memoria	CORRECTO
6	Recuperar proveedor recordado	respuesta_general	CORRECTO
7	Generar reporte de porcelanatos	generar_reporte	CORRECTO

3.2 Evidencia de ejecución

Indicador	Resultado
Archivos cargados	2
Fragmentos indexados	16
Fuentes disponibles	2
Turnos de memoria corta	6
Datos en memoria persistente	1
Reportes escritos	1
Estado del agente	Compilado correctamente

Las pruebas utilizaron PROVEEDORES REMODELACIONES.xlsx y REMODELACION CABAÑA SOL.xlsx. El sistema mantuvo el origen de los fragmentos y recuperó información desde ambos archivos según la consulta.

4. Documentación técnica y entrega

El código fuente se entrega en un repositorio GitHub/GitLab junto con el notebook ejecutado, los archivos de prueba autorizados, el README y el diagrama de orquestación. El README debe indicar dependencias, configuración de GOOGLE_API_KEY en Secrets, formatos admitidos, orden de ejecución y descripción breve de los componentes.

Repositorio: https://github.com/MLoyolaC/agente-omm-hotel-el-almendro

4.1 Registro del uso de IA

Google Gemini API se utilizó como componente funcional del agente para generación de texto, decisiones estructuradas y embeddings. ChatGPT se utilizó como apoyo para revisión de redacción, organización del material y preparación de diagramas/documentación. Las salidas fueron revisadas y validadas por el equipo antes de incorporarlas al proyecto.

4.2 Justificación técnica del equipo

Inicialmente intentamos utilizar Mistral, siguiendo los ejemplos trabajados en clases y el repositorio de la asignatura, pero tuvimos problemas con los límites de uso de la API, por lo que decidimos utilizar Gemini, ya que nos permitió continuar el desarrollo de forma más sencilla. Para la selección de herramientas nos guiamos principalmente por los ejercicios vistos en clases y los notebooks del repositorio https://github.com/michael-palomino-tm/Ingenier-a-de-Soluciones-con-Inteligencia-Artificial. A partir de esos ejemplos implementamos LangGraph para organizar el flujo y la toma de decisiones del agente, embeddings para recuperar información relevante desde los documentos, memoria de corto y largo plazo para mantener contexto entre consultas y herramientas separadas para buscar, analizar, guardar información y generar reportes. Elegimos LangGraph porque nos permite ordenar mejor el flujo del agente, manejar la memoria y decidir qué herramienta usar según cada consulta. Para los Excel procesamos la información por filas y hojas manteniendo sus encabezados, para que datos como proveedor, producto o precio no perdieran su contexto al momento de realizar la búsqueda semántica.

5. Conclusiones y reflexiones individuales

Matías Loyola: Logré entender mejor cómo se procesa la información de los archivos, ya que primero se cargan y extraen sus datos, luego se generan embeddings para buscar el contexto más relacionado con la pregunta, y finalmente ese contexto se entrega a Gemini para generar una respuesta basada en los documentos.

Matías Cisternas: Aprendí cómo el agente utiliza distintas herramientas según la consulta y cómo la memoria permite mantener información entre preguntas. También entendí mejor la función del planificador para decidir qué acción ejecutar. Mi aporte estuvo principalmente en probar los distintos casos y revisar que el flujo y las respuestas funcionaran correctamente.

Referencias

Google. (2026). Gemini API documentation. https://ai.google.dev/gemini-api/docs

LangChain. (2026). LangGraph documentation. https://docs.langchain.com/oss/python/langgraph/overview

Palomino, M. (2026). Ingeniería de Soluciones con Inteligencia Artificial [Repositorio GitHub]. GitHub. https://github.com/michael-palomino-tm/Ingenier-a-de-Soluciones-con-Inteligencia-Artificial

OpenAI. (2026). ChatGPT [Modelo de IA generativa]. https://chatgpt.com/